

Student: Jakub Kwaśniewski

Numer albumu 32430

Sprawozdanie: 06 grafy

Metody Programowania

Uczelnia: Politechnika Morska w Szczecinie

Data: 31.05.2026

### Wstęp do zadania:

Celem ćwiczenia było zapoznanie się ze strukturą danych w postaci grafów oraz ich praktyczną implementacją w języku C++. W ramach zadania przeanalizowano dwa sposoby reprezentacji grafu macierz sąsiedztwa oraz listę sąsiedztwa, a następnie przetestowano działanie metod odpowiedzialnych za dodawanie krawędzi i wyświetlanie struktury. Wykonano również rozbudowę kodu o interfejs umożliwiający użytkownikowi wybór sposobu reprezentacji nadawanie własnych nazw wierzchołkom oraz ręczne tworzenie połączeń między nimi.

Zadanie 1. Jakie jest rozwiązanie problemu, który stawiali sobie mieszkańcy Królewca? Uzasadnij odpowiedź.

Spacer jest niemożliwy, ponieważ każda z czterech części lądu ma nieparzystą liczbę mostów, a według reguły Eulera, aby przejść całą trasę bez powtarzania mostów miejsca z nieparzystą liczbą połączeń mogą być maksymalnie dwa.

Zadanie 3 Przepisz powyższe fragmenty kodu. W funkcji main() utwórz grafy za pomocą obu klas i wykonaj testy każdej metody [addEdge, print]. Następnie opisz działanie każdej metody.

```
setlocale(LC_CTYPE, "Polish");
cout << "Matrix" << endl;
// Tworzymy graf o 3 wierzchołkach
GraphMatrix gm(3);

// Dodaje krawędzie
gm.addEdge(0, 1); // Połączenie między 0 a 1
gm.addEdge(1, 2); // Połączenie między 1 a 2

//Wyświetlenie macierzy
gm.print();

cout << "Neighborhood Lists" << endl;

GraphList gl(3);

// Dodaje krawędzie
gl.addEdge(0, 1); // Połączenie między 0 a 1;
gl.addEdge(1, 2); // Połączenie między 1 a 2;

// Wyświetlanie listy
gl.print();
```

```
Matrix
0 1 0
1 0 1
0 1 0
Neighborhood Lists
0: 1
1: 0 2
2: 1
```

```
class GraphMatrix {
private:
    std::vector<std::vector<int>> graphMatrix;
    int verticesNumber;
```

class GraphMatrix - Rozpoczęcie klasy.

std::vector<std::vector<int>> graphMatrix; - Deklaracja dwuwymiarowego wektora, który będzie przechowywał informację o połączeniach.

int verticesNumber; - Zmienna całkowita, która zapamięta ile wierzchołków ma graf.

```
GraphMatrix(int vertices) {
    verticesNumber = vertices;
    graphMatrix.resize(vertices, std::vector<int>(vertices, 0));
}
```

GraphMatrix(int vertices) – Konstruktor klasy.

verticesNumber = vertices; - Przypisanie podanej w argumencie liczby wierzchołków do zmiennej klasy.

graphMatrix.resize(vertices, std::vector<int>(vertices, 0)); - Zmiana rozmiaru tabeli na kwadrat i wypełnienie wszystkich jej komórek zerami.

```
void addEdge(int i, int j) {
    graphMatrix[i][j] = 1;
    graphMatrix[j][i] = 1; // usuń, jeśli graf ma być skierowany
}
```

void addEdge(int i, int j)- Definicja metody.

graphMatrix[i][j] = 1;- Wypisanie jedynki na przecięciu wiersza i oraz kolumny j.

graphMatrix[j][i] = 1; - wypisanie jedynki na przecięciu wiersza j oraz kolumny i.

```
void print() {
    for (int i = 0; i < verticesNumber; ++i) {
        for (int j = 0; j < verticesNumber; ++j) {
            std::cout << graphMatrix[i][j] << " ";
        }
        std::cout << "\n";
    }
}
```

void print() – Definicja metody.

for (int i = 0; i < verticesNumber; ++i) – Główna pętla for przechodzi po kolei przez każdy wiersz tabeli.

for (int j = 0; j < verticesNumber; ++j) – Wewnętrzna pętla for przechodzi po kolei przez każdą kolumnę w danym wierszy.

std::cout << graphMatrix[i][j] << " "; - Wypisanie na ekran wartości z komórki.

std::cout << "\n"; - Przejście do nowej linii.

```
class GraphList {
private:
    std::vector<std::vector<int>> graphList;
public:
```

class GraphList – Rozpoczęcie klasy.

std::vector<std::vector<int>> graphList; - Deklaracja dwuwymiarowego wektora, gdzie każdy element to osobna lista przechowująca numery sąsiednie danego wierzchołka.

```
GraphList(int vertices) {
    graphList.resize(vertices);
}
```

GraphList(int vertices) – Konstruktor klasy przyjmujący jako parametr liczbę całkowitą.

```
void addEdge(int u, int v) {
    graphList[u].push_back(v);
    graphList[v].push_back(u); // usuń, jeśli graf ma być skierowany
}
```

void addEdge(int u, int v) – Definicja metody. Dodaje krawędź między wierzchołkami u oraz v.

graphList[u].push\_back(v); - Otwarcie listy Przypisanej do wierzchołka u i dopisanie na jej koniec numeru v.

graphList[v].push\_back(u); - Otwarcie listy wierzchołka v i dopisanie na jej koniec numeru u.

```
void print() {
    for (int i = 0; i < graphList.size(); ++i) {
        std::cout << i << ": ";
        for (int neighbor : graphList[i]) {
            std::cout << neighbor << " ";
        }
        std::cout << "\n";
    }
}
```

void print() – definicja metody.

for (int i = 0; i < graphList.size(); ++i) – Pętla for przechodzi po kolei każdy wierzchołek w grafie.

std::cout << i << ": "; - Wypisanie numeru aktualnego wierzchołka.

for (int neighbor : graphList[i]) – Pętla przechodząca bezpośrednio przez wszystkie liczby zapisane na liście sąsiadów wierzchołka i.

std::cout << neighbor << " "; - Wypisanie na ekran numeru każdego znalezionej sąsiada.

std::cout << "\n"; - Przejście do nowej linii.

Zadanie 4 Stwórz interfejs umożliwiający użytkownikowi programu:

a. Wybór sposobu reprezentacji grafu;

```
// Wybór sposobu reprezentacji grafu
int choice;
cout << "Choose graph representation " << endl;
cout << "1- Adjacency Matrix " << endl;
cout << "2- Adjacency List " << endl;
cout << "Your choice: ";
cin >> choice;
```

```
Choose graph representation
1- Adjacency Matrix
2- Adjacency List
Your choice: |
```

b. Wybór liczby wierzchołków;

```
// Wybór liczby wierzchołków
int vertexCount;
cout << "Enter the number of vertices: ";
cin >> vertexCount;
```

```
Enter the number of vertices: 1
```

c. Wpisanie nazw wierzchołków;

```
// Wpisanie nazw wierzchołków
vector<string> vertexNames(vertexCount);
cout << "Enter the name for each vertex: " << endl;
for (int i = 0; i < vertexCount; ++i) {
    cout << "Vertex " << i << ": ";
    cin >> vertexNames[i];
}

// Tworzenie obiektów obu klas
GraphMatrix gmm(vertexCount);
GraphList gll(vertexCount);
```

```
Enter the number of vertices: 3
Enter the name for each vertex:
Vertex 0: one
Vertex 1: two
Vertex 2: three
```

d. Stworzenie grafu, czyli możliwość wpisania przez użytkownika, który wierzchołek ma połączenie, z którym.

```
// Stworzenie grafu
int edgeCount;
cout << "How many edges do you want to add?: ";
cin >> edgeCount;

cout << "Enter edges by typing two vertex names: " << endl;
for (int k = 0; k < edgeCount; ++k) {
    string start, end;
    cout << "Edge " << k + 1 << ": ";
    cin >> start >> end;
    // Szukanie indeksu liczbowego dla wpisanej nazwy start
    int idStart = -1;
    for (int i = 0; i < vertexCount; ++i) {
        if (vertexNames[i] == start) {
            idStart = i;
            break;
        }
    }
}
```

```
// szukanie indeksu liczbowego dla wpisanej nazwy end
int idEnd = -1;
for (int i = 0; i < vertexCount; ++i) {
    if (vertexNames[i] == end) {
        idEnd = i;
        break;
    }
}

// jeśli obie nazwy zostały znalezione
if (idStart != -1 && idEnd != -1) {
    if (choice == 1) {
        gmm.addEdge(idStart, idEnd);
    }
    else
    {
        gll.addEdge(idStart, idEnd);
    }
}
```

```
    else {
        // jeśli użytkownik wpisał złą nazwę
        cout << "ERROR- Vertex name does not exist" << endl;
        k--;
    }
}

// Wyświetlenie
if (choice == 1) {
    gmm.print();
}
else {
    gll.print();
}
```



Zadanie 5. Ulepsz metodę print w obu klasach. Ulepszenie ma polegać na tym, że wyświetlane będą nazwy wierzchołków, co zwiększy czytelność obu reprezentacji.

```
void print(const vector<string>& names) {
    cout << "List" << endl;
    for (int i = 0; i < graphList.size(); ++i) {
        std::cout << names[i] << ": ";
        for (int neighbor : graphList[i]) {
            std::cout << names[neighbor] << " ";
        }
        std::cout << "\n";
    }
}

void print(const vector<string>& names) {
    cout << "Matrix" << endl;
    for (const string& name : names)
        cout << name << " " << endl;

    for (int i = 0; i < verticesNumber; ++i) {
        for (int j = 0; j < verticesNumber; ++j) {
            std::cout << graphMatrix[i][j] << " ";
        }
        std::cout << "\n";
    }
}
```

Choose graph representation

1- Adjacency Matrix

2- Adjacency List

Your choice: 1

Enter the number of vertices: 2

Enter the name for each vertex:

Vertex 0: one

Vertex 1: two

How many edges do you want to add?: 1

Enter edges by typing two vertex names:

Edge 1: one

two

Matrix

one

two

0 1

1 0

## Wnioski:

Wybór sposobu zapisu grafu zależy od liczby jego połączeń. Macierz sąsiedztwa sprawdza się najlepiej dla grafów gęstych z dużą liczbą krawędzi, ponieważ pozwala na sprawdzenie czy dwa punkty są ze sobą połączone. Lista sąsiedztwa jest bardziej oszczędna pod względem pamięci komputera dla grafów rzadkich, ponieważ zapisuje wyłącznie istniejące relacje i ignoruje brak połączeń. Dodatkowo zastąpienie domyślnych numerów tekstem z nazwami wierzchołków znacząco poprawia czytelność programu.